

PYTHON & GenerativeAI

A Comprehensive Training Manual

Core Python | FastAPI | Generative AI | LangChain

3-Day Intensive Course

Day 1: Core Python Fundamentals

Day 2: REST API Development with FastAPI

Day 3: Generative AI, Prompt Engineering & LangChain

Suitable for: Academic, Corporate & Professional Training Environments

Table of Contents

Preface

This training manual is a comprehensive resource compiled from a structured 3-day intensive course covering Python programming and Artificial Intelligence. It is designed for learners ranging from beginners to intermediate-level developers who wish to gain practical, industry-relevant skills in Python and modern AI development.

The manual is organized into three major modules:

- Day 1 — Core Python Fundamentals: Variables, data types, control flow, data structures, functions, and lambda expressions.
- Day 2 — REST API Development with FastAPI: Building production-ready APIs, CRUD operations, request handling, status codes, and Swagger UI.
- Day 3 — Generative AI and LangChain: Understanding AI concepts, prompt engineering techniques, calling LLMs from code, and building intelligent applications using LangChain.

Each chapter includes detailed theoretical explanations, annotated code examples, real-world use cases, summary tables, and a comprehensive set of exercises including short-answer, long-answer, scenario-based, MCQ, and interview questions.

This document is intended to serve as a standalone reference that can be used during training sessions, for self-study, or as a professional handbook on the desk of any Python or AI developer.

MODULE 1

Day 1: Core Python Fundamentals

Data Structures, Functions & Practice Programs

Chapter 1: Variables in Python

1.1 Introduction to Variables

In Python, a variable is a named storage location in memory that holds a value. Unlike many other programming languages, Python does not require the programmer to explicitly declare the data type of a variable. Instead, Python uses a concept known as dynamic typing, where the interpreter automatically infers the type of the variable based on the value assigned to it.

Variables are fundamental to programming because they allow a program to store, retrieve, and manipulate data dynamically. Without variables, every computation would need to use literal values directly, making programs rigid and impractical.

1.2 Variable Naming Rules

Python enforces specific rules for naming variables. Violating these rules will result in a syntax error:

- Variable names must start with a letter (a–z, A–Z) or an underscore (_).
- They can contain letters, digits (0–9), and underscores, but not spaces or special characters.
- Variable names are case-sensitive: `age`, `Age`, and `AGE` are three distinct variables.
- Reserved keywords (such as `if`, `for`, `while`, `class`, `return`) cannot be used as variable names.

Best Practice

Use lowercase letters and underscores for variable names (e.g., `student_name`, `total_price`). This is known as `snake_case` and is the standard Python naming convention as defined by PEP 8.

1.3 Variable Assignment Syntax

The assignment operator `=` is used to assign a value to a variable. The syntax is:

```
variable_name = value
```

Python also supports multiple assignment in a single line:

```
# Assign same value to multiple variables
x = y = z = 0

# Assign different values in one line
name, age, price = "Alice", 22, 99.5
```

1.4 Practical Examples

```
name = "Alice"      # String variable
age  = 22           # Integer variable
price = 99.5        # Float variable
is_active = True    # Boolean variable

print(name)         # Output: Alice
print(age)          # Output: 22
print(price)        # Output: 99.5
```

1.5 Real-World Use Cases

- E-commerce: Storing product prices, customer names, and stock quantities.
- Healthcare: Storing patient age, blood group, and diagnosis codes.
- Finance: Holding account balances, interest rates, and transaction amounts.
- Education portals: Storing student roll numbers, subject names, and grades.

1.6 Chapter Summary

Concept	Description
Variable	Named container that stores a value in memory
Dynamic Typing	Python infers data type automatically at runtime
Assignment	Use the = operator to assign a value to a variable
snake_case	Recommended naming style: lowercase with underscores

Chapter 2: Python Data Types

2.1 Overview

A data type defines the kind of value a variable can hold and the operations that can be performed on it. Python provides a rich set of built-in data types that cover virtually every kind of data a programmer might work with.

Python's type system is dynamic — the type of a variable is determined at runtime based on the assigned value. However, Python is also strongly typed, meaning implicit conversion between incompatible types is not allowed.

2.2 Core Data Types Reference Table

Data Type	Description	Example Values	Use Case
int	Whole numbers, positive or negative	10, -5, 0, 1000	Counters, ages, IDs
float	Numbers with a decimal point	3.14, 10.5, -0.001	Prices, measurements, GPS
str	Sequence of text characters	"Hello", 'Python'	Names, messages, labels
bool	Logical True or False value	True, False	Flags, conditions, switches
list	Ordered, mutable collection	[1, 2, 3], ["a", "b"]	Shopping carts, task lists
tuple	Ordered, immutable collection	(1, 2, 3), ("x", "y")	Coordinates, RGB values
set	Unordered unique values	{1, 2, 3}	Removing duplicates, math
dict	Key-value pairs	{"name": "Meet"}	JSON data, records, configs

2.3 Type Checking and Conversion

Python provides the `type()` function to check the type of a variable at runtime. For converting between types, Python offers built-in conversion functions:

```
x = 10
print(type(x))           # <class 'int'>
```

```
# Type Conversion (Casting)
y = float(x)          # int -> float: 10.0
z = str(x)            # int -> str: '10'
a = int("25")         # str -> int: 25
b = bool(0)           # int -> bool: False
c = list((1, 2, 3))   # tuple -> list: [1, 2, 3]
```

2.4 Mutable vs Immutable Types

An important distinction in Python is between mutable and immutable types. A mutable object can be changed after it is created, while an immutable object cannot.

Category	Data Types	Can be Changed?
Mutable	list, set, dict	Yes — elements can be added, removed, or updated
Immutable	int, float, str, tuple, bool	No — a new object must be created for any change

2.5 Chapter Summary

Python's built-in data types provide a powerful and flexible foundation for data storage and manipulation. Choosing the correct data type for a given task is critical for writing efficient, correct, and readable code. The key points are: use int and float for numerical data, str for text, bool for logic, and the collection types (list, tuple, set, dict) for grouping related values.

Chapter 3: Control Flow in Python

3.1 The For Loop

A for loop is a control flow statement that repeats a block of code for a fixed number of iterations. In Python, for loops are most commonly used with the built-in `range()` function or by iterating directly over a collection (list, string, tuple, etc.).

3.1.1 Syntax

```
for variable in range(start, stop, step):  
    # code block
```

The `range()` function generates a sequence of numbers. Key parameters:

- **start:** The number to begin at (inclusive). Default is 0.
- **stop:** The number to end before (exclusive). Required.
- **step:** The increment (or decrement) between each number. Default is 1.

3.1.2 Practical Examples

```
# Basic range loop  
for i in range(1, 6):  
    print(i)          # Prints 1 through 5  
  
# Loop with step  
for i in range(0, 10, 2):  
    print(i)          # Prints 0, 2, 4, 6, 8  
  
# Iterating over a list  
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)  
  
# Enumerate for index + value  
for index, fruit in enumerate(fruits):  
    print(f"{index}: {fruit}")
```

3.2 The if, if-else, and if-elif-else Statements

Conditional statements allow a program to make decisions and execute different blocks of code depending on whether a condition evaluates to True or False.

3.2.1 The if Statement

The simplest form of conditional logic. A block of code executes only if the given condition is True.

```
age = 18
if age >= 18:
    print("Eligible to vote")
```

3.2.2 The if-else Statement

Provides an alternative block of code that runs when the condition is False.

```
num = 7
if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

3.2.3 The if-elif-else Statement

Used when multiple conditions need to be evaluated sequentially. The first True condition's block is executed, and the rest are skipped.

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Grade D")
```

3.3 Real-World Applications of Control Flow

- Banking: Checking if account balance is sufficient before processing a withdrawal.
- E-commerce: Applying different discount tiers based on purchase amount.
- Authentication systems: Granting different access levels based on user roles.
- Report generation: Printing all records in a loop, filtering by condition.

Chapter 4: Python Data Structures

Python provides four primary built-in collection data structures: List, Tuple, Set, and Dictionary. Each has unique characteristics that make it suitable for different types of tasks. Additionally, Strings are treated as ordered, immutable sequences of characters and are covered in this chapter.

4.1 Lists

4.1.1 Overview

A list is an ordered, mutable (changeable) collection of items that allows duplicate values and supports mixed data types. Lists are the most versatile and widely used data structure in Python.

- Created using square brackets: `my_list = [1, 2, 3]`
- Supports indexing (positive and negative) and slicing.
- Can store elements of any type, including other lists (nested lists).
- Mutable: elements can be added, removed, or modified after creation.

4.1.2 List Methods Reference

Method	Description	Example
<code>append(x)</code>	Add element at end	<code>lst.append(5)</code>
<code>insert(i, x)</code>	Insert at specific index <code>i</code>	<code>lst.insert(2, 'a')</code>
<code>remove(x)</code>	Remove first occurrence of value <code>x</code>	<code>lst.remove(3)</code>
<code>pop(i)</code>	Remove and return element at index <code>i</code>	<code>lst.pop(0)</code>
<code>clear()</code>	Remove all items from list	<code>lst.clear()</code>
<code>index(x)</code>	Return index of first occurrence of <code>x</code>	<code>lst.index('a')</code>
<code>count(x)</code>	Count how many times <code>x</code> appears	<code>lst.count(2)</code>
<code>sort()</code>	Sort the list in ascending order (in-place)	<code>lst.sort()</code>
<code>reverse()</code>	Reverse the list order (in-place)	<code>lst.reverse()</code>
<code>copy()</code>	Return a shallow copy of the list	<code>lst2 = lst.copy()</code>

4.1.3 List Slicing

Slicing allows extracting a portion of a list using the [start:stop:step] syntax.

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4])    # [1, 2, 3]    - index 1 to 3
print(nums[:2])     # [0, 2, 4]    - every 2nd element
print(nums[::-1])   # [5, 4, 3, 2, 1] - reversed
```

4.1.4 Real-World Use Cases

- Shopping Cart: Maintain an ordered list of items added by a user.
- Task Management: Store pending to-do items; append new tasks, pop completed ones.
- Data Processing: Collect user inputs in a loop, sort and filter before saving.

4.2 Tuples

4.2.1 Overview

A tuple is an ordered, immutable collection of items. Once created, its elements cannot be modified. This property makes tuples ideal for storing data that should not change throughout the lifetime of a program, such as geographic coordinates, RGB color codes, or database connection parameters.

- Created using parentheses: `my_tuple = (1, 2, 3)`
- Immutable: elements cannot be added, removed, or changed after creation.
- Allows duplicate values and mixed data types.
- Faster than lists for read-only operations due to immutability.

4.2.2 Tuple Methods

Method	Description	Example
<code>count(x)</code>	Count how many times x appears in tuple	<code>t.count(2)</code> # returns 3
<code>index(x)</code>	Return the index of first occurrence of x	<code>t.index('a')</code> # returns 2

4.2.3 Tuple Unpacking

Tuple unpacking allows assigning tuple elements to multiple variables in a single statement — a powerful Python feature.

```
coords = (10, 20)
x, y = coords
print(x, y)          # 10 20

# Extended unpacking with *
a, *rest = (1, 2, 3, 4)
print(a, rest)       # 1 [2, 3, 4]
```

4.3 Sets

4.3.1 Overview

A set is an unordered, mutable collection of unique items. Sets automatically remove duplicate values and support mathematical set operations such as union, intersection, and difference. Sets do not support indexing or slicing because their elements have no fixed position.

- Created with curly braces: `my_set = {1, 2, 3}`
- All elements must be unique (duplicates are silently ignored on creation).
- Mutable: elements can be added and removed.
- Unordered: no guarantee of element ordering.

4.3.2 Set Methods Reference

Method	Description
<code>add(x)</code>	Add a single item to the set
<code>update(iter)</code>	Add multiple items from an iterable
<code>remove(x)</code>	Remove item <code>x</code> ; raises <code>KeyError</code> if not found
<code>discard(x)</code>	Remove item <code>x</code> safely (no error if not found)
<code>pop()</code>	Remove and return an arbitrary item
<code>clear()</code>	Remove all items from set
<code>union(s)</code>	Return new set with all items from both sets
<code>intersection(s)</code>	Return new set with only common items
<code>difference(s)</code>	Return items in this set but not in <code>s</code>
<code>symmetric_difference(s)</code>	Return items not common to both sets
<code>issubset(s)</code>	Check if this set is a subset of <code>s</code>
<code>issuperset(s)</code>	Check if this set is a superset of <code>s</code>

4.3.3 Set Operations Example

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}

print(s1 | s2)    # Union:      {1,2,3,4,5,6}
print(s1 & s2)    # Intersection: {3,4}
print(s1 - s2)    # Difference:  {1,2}
print(s1 ^ s2)    # Symmetric:  {1,2,5,6}
```

```
# Removing duplicates from a list using a set
lst = [1, 2, 2, 3, 3, 4]
unique = list(set(lst))
print(unique)      # [1, 2, 3, 4]
```

4.4 Dictionaries

4.4.1 Overview

A dictionary is a collection of key-value pairs. Each key must be unique and immutable (typically a string or number), while values can be of any type and can repeat. Dictionaries are ordered (since Python 3.7+), mutable, and provide extremely fast lookups by key.

Dictionaries are the foundation of Python's internal object model and are used extensively in web APIs, configuration files (JSON format), and data processing pipelines.

- Created with curly braces: `my_dict = {"name": "Meet", "age": 20}`
- Access values using their keys: `my_dict["name"]`
- Keys must be unique; duplicate keys overwrite earlier values.
- Values can be of any data type, including nested dictionaries.

4.4.2 Dictionary Methods Reference

Method	Description
<code>keys()</code>	Returns a view of all keys in the dictionary
<code>values()</code>	Returns a view of all values
<code>items()</code>	Returns a view of all (key, value) pairs
<code>get(key, default)</code>	Safely get value; returns default if key not found
<code>update(dict2)</code>	Merge another dictionary into this one
<code>pop(key)</code>	Remove key and return its value
<code>clear()</code>	Remove all key-value pairs
<code>copy()</code>	Return a shallow copy of the dictionary

4.4.3 Dictionary Comprehension

```
# Create a dictionary from scratch using comprehension
squares = {x: x**2 for x in range(1, 6)}
print(squares)      # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Iterate over a dictionary
student = {"name": "Alice", "age": 21, "grade": "A"}
```

```
for key, value in student.items():
    print(f"{key}: {value}")
```

4.5 Strings

4.5.1 Overview

A string is an ordered, immutable sequence of characters. Strings are one of the most commonly used data types in Python and come with a rich set of built-in methods for manipulation. Strings can be created using single, double, or triple quotes.

4.5.2 String Methods Reference

Method	Description
upper() / lower()	Convert to all uppercase or all lowercase
title() / capitalize()	Title Case or Capitalize first character
strip() / lstrip() / rstrip()	Remove whitespace from both/left/right ends
replace(old, new)	Replace all occurrences of a substring
split(sep)	Split string into a list by separator
join(iter)	Join elements of an iterable into a string
find(sub)	Return index of first occurrence; -1 if not found
startswith() / endswith()	Check if string begins or ends with substring
count(sub)	Count non-overlapping occurrences of sub

4.5.3 F-Strings (Formatted String Literals)

Introduced in Python 3.6, f-strings provide a concise and readable way to embed expressions inside string literals. They are the recommended approach for string formatting in modern Python.

```
name = "Bob"
age = 25
print(f"Name: {name}, Age: {age}")
# Output: Name: Bob, Age: 25

# Expressions inside f-strings
price = 99.5
print(f"Price with GST: {price * 1.18:.2f}")
# Output: Price with GST: 117.41
```

4.6 Comparison of Data Structures

Data Type	Ordered	Mutable	Duplicates	Syntax
List	Yes	Yes	Allowed	[1, 2, 3]
Tuple	Yes	No	Allowed	(1, 2, 3)
Set	No	Yes	Not Allowed	{1, 2, 3}
Dictionary	Yes (3.7+)	Yes	Keys: No, Values: Yes	{"key": "val"}
String	Yes	No	Allowed	"Hello"

Chapter 5: Functions and Lambda Expressions

5.1 Introduction to Functions

A function is a named, reusable block of code that is designed to perform a specific task. Functions are the primary building blocks for organizing Python programs into modular, maintainable components.

5.1.1 Why Use Functions?

- **Code Reusability:** Write once, call many times — eliminates repetition.
- **Modularity:** Break complex programs into smaller, manageable parts.
- **Readability:** Self-documenting code with meaningful function names.
- **Easier Debugging:** Isolate and fix problems within individual functions.
- **Abstraction:** Hide complex implementation details behind a simple interface.

5.1.2 Function Syntax

```
def function_name(parameter1, parameter2, ...):  
    """Optional docstring describing the function."""  
    # code block  
    return value    # Optional
```

5.1.3 Examples

```
# Basic function with return  
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("Alice"))    # Hello, Alice!  
  
# Function with default parameter  
def power(base, exp=2):  
    return base ** exp  
  
print(power(3))          # 9   (exp defaults to 2)  
print(power(2, 10))      # 1024
```

5.2 Lambda Functions

A lambda function is a small, anonymous (unnamed) function that can have multiple arguments but contains only a single expression. Lambdas are defined using the lambda keyword and are typically used for short, throwaway operations where creating a full named function would be unnecessarily verbose.

5.2.1 Syntax

```
lambda arguments: expression
```

5.2.2 Lambda vs Regular Function

Feature	Regular Function (def)	Lambda Function
Name	Has a name	Anonymous (no name)
Lines	Can span multiple lines	Single expression only
Return	Explicit return statement	Expression result returned implicitly
Reusability	Can be called anywhere by name	Typically used inline
Use Case	Complex logic, reusable code	Short operations, map/filter/sorted

5.2.3 Lambda with map() and filter()

```
nums = [1, 2, 3, 4, 5]

# map() - apply function to each element
squares = list(map(lambda x: x ** 2, nums))
print(squares)    # [1, 4, 9, 16, 25]

# filter() - keep elements where function returns True
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)      # [2, 4]

# sorted() with lambda key
students = [("Alice", 85), ("Bob", 72), ("Charlie", 90)]
by_score = sorted(students, key=lambda s: s[1], reverse=True)
print(by_score)   # [('Charlie', 90), ('Alice', 85), ('Bob', 72)]
```

5.3 *args and **kwargs

Python supports variable-length arguments using *args and **kwargs. These are powerful features that make functions flexible enough to accept any number of arguments.

5.3.1 *args — Variable Positional Arguments

When a function is defined with `*args`, it can accept any number of positional arguments. Inside the function, `args` is a tuple containing all the extra positional arguments.

```
def total(*nums):  
    return sum(nums)  
  
print(total(1, 2, 3, 4))    # 10  
print(total(5, 10))        # 15
```

5.3.2 **kwargs — Variable Keyword Arguments

When a function is defined with `**kwargs`, it can accept any number of keyword arguments. Inside the function, `kwargs` is a dictionary containing all the extra keyword arguments.

```
def info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
info(name="Bob", age=22, city="Ahmedabad")  
# Output:  
# name: Bob  
# age: 22  
# city: Ahmedabad
```

5.4 Chapter Summary

Functions are the cornerstone of clean, maintainable Python code. Regular functions (`def`) are used for complex, reusable logic; lambda functions are used for concise, inline operations. The `*args` and `**kwargs` mechanisms provide tremendous flexibility for building general-purpose utilities and APIs.

Chapter 6: Practice Questions and Exercises — Day 1

6.1 Short Answer Questions

1. What is dynamic typing in Python? How does it differ from static typing?
2. List any four rules that must be followed when naming variables in Python.
3. What is the difference between mutable and immutable data types? Give one example of each.
4. What is the output of `range(2, 10, 3)`? List all values.
5. What is the difference between `remove()` and `discard()` in a Python set?
6. How does a dictionary differ from a list in Python?
7. What is a lambda function? When would you prefer it over a regular function?
8. Explain the difference between `*args` and `**kwargs`.
9. What does the `get()` method of a dictionary do? Why is it preferred over direct key access?
10. What is the output of `[1, 2, 3, 4, 5][::2]`?

6.2 Long Answer Questions

11. Explain all Python built-in data types with examples, advantages, and ideal use cases for each. Include a comparison table.
12. Describe all the methods available for Python lists with syntax, description, and working examples for each method.
13. Compare and contrast Lists, Tuples, Sets, and Dictionaries in Python. Include a table summarizing their properties. Explain when you would choose each data structure in a real application.
14. Explain the concept of slicing in Python. Provide examples of forward slicing, backward slicing, step slicing, and reversing a list.
15. Explain Python functions in detail: definition, syntax, types of parameters (positional, default, `*args`, `**kwargs`), return values, and scope. Support each concept with code examples.

6.3 Practical / Coding Questions

16. Write a Python program that takes a list of numbers and prints the sum, average, minimum, and maximum without using built-in functions other than `len()`.
17. Write a function that takes a sentence as input and returns a dictionary where each word is a key and its length is the value.
18. Write a Python program to remove duplicate values from a list while preserving the original order.

19. Given a list of student dictionaries [{"name": "Alice", "marks": 85}, ...], sort the list by marks in descending order using a lambda function.
20. Write a function using *args that accepts any number of grades and returns the letter grade (A/B/C/D/F) based on the average.
21. Write a Python program that counts the frequency of each character in a given string using a dictionary.
22. Create a program that uses a set to find all words that appear in both of two given sentences.
23. Write a function that accepts a dictionary of student names and their scores, and returns a new dictionary containing only students who scored above 75.

6.4 Multiple Choice Questions

24. Which of the following data types is immutable in Python? a) list b) dict c) set d) tuple
Answer: d) tuple
25. What is the output of len({1, 2, 2, 3, 3, 4})? a) 6 b) 5 c) 4 d) 3 Answer: c) 4
26. Which method adds an element to the end of a list? a) add() b) append() c) insert() d) push() Answer: b) append()
27. What does the pop() method return when called on a list with no arguments? a) The first element b) The middle element c) The last element d) Nothing Answer: c) The last element
28. Which is the correct syntax for a lambda function that doubles a value? a) lambda x => x*2 b) lambda x: x*2 c) def lambda(x): x*2 d) lambda(x): return x*2 Answer: b) lambda x: x*2
29. What is the output of 'Hello World'.split()? a) ['Hello World'] b) ['H','e','l','l','o',' ','W','o','r','l','d'] c) ['Hello', 'World'] d) Error Answer: c) ['Hello', 'World']

6.5 Scenario-Based Questions

30. Scenario: You are building a student grade management system. Which data structure would you use to store each student's information (name, roll number, marks, grade)? Justify your choice and write a code snippet demonstrating the structure.
31. Scenario: An e-commerce platform receives a list of product IDs from two different warehouses. You need to find: (a) all unique products across both, (b) products available in both, and (c) products only in one warehouse. Which data structure and which operations would you use? Write the code.
32. Scenario: You need to build a function that generates a personalized report. The function should accept a name and any number of key-value details (department, date, score, etc.). Design this function and demonstrate calling it with different numbers of keyword arguments.

6.6 Interview Questions

33. What is the difference between is and == in Python?

34. Explain Python's memory management and garbage collection in simple terms.
35. What is a shallow copy vs a deep copy? How do you perform each in Python?
36. What is list comprehension? Give an example and explain when to use it.
37. How does Python's `sort()` differ from `sorted()`? Which modifies the original list?
38. What happens when you try to use a list as a dictionary key? Why?
39. Explain the concept of scope in Python (LEGB rule).

MODULE 2

Day 2: Building REST APIs with FastAPI

CRUD Operations, Request Handling, Validation & Documentation

Chapter 7: Introduction to FastAPI

7.1 What is an API?

An Application Programming Interface (API) is a set of rules and protocols that allows different software applications to communicate with each other. A REST API (Representational State Transfer) is a standard architectural style for building web APIs using HTTP methods. REST APIs are the backbone of modern web applications, enabling frontend clients, mobile apps, and other backend services to exchange data.

7.2 What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building RESTful APIs. It was created by Sebastián Ramírez and released in 2018. FastAPI is built on two powerful libraries: Starlette (for the web server infrastructure) and Pydantic (for data validation and serialization using Python type hints).

7.2.1 Key Features of FastAPI

- **Extremely Fast:** Performance comparable to NodeJS and Go — one of the fastest Python frameworks available.
- **Automatic Data Validation:** Uses Python type hints and Pydantic to automatically validate incoming request data.
- **Auto-Generated Documentation:** Swagger UI (at /docs) and ReDoc (at /redoc) are generated automatically.
- **Asynchronous Support:** Natively supports async/await for non-blocking I/O operations.
- **Type Safety:** Type hints make the code self-documenting and reduce bugs.
- **Easy to Learn:** Minimal boilerplate; clean, intuitive API design.

7.2.2 FastAPI vs Other Python Frameworks

Feature	FastAPI	Flask	Django REST
Performance	Very High (async)	Moderate	Moderate
Type Validation	Automatic (Pydantic)	Manual	Serializers required
Auto Docs	Built-in (Swagger)	Not built-in	Not built-in
Learning Curve	Low	Very Low	High
Async Support	Native	Limited	Limited

Feature	FastAPI	Flask	Django REST
Best For	APIs, microservices	Small apps, prototypes	Full-stack web apps

7.3 Installation and Project Setup

FastAPI requires three packages to be installed:

```
pip install fastapi      # The FastAPI framework
pip install uvicorn      # ASGI server to run the app
pip install pydantic     # Data validation library
```

To start the server:

```
python -m uvicorn main:app --reload
```

The `--reload` flag enables auto-reload on file save — extremely useful during development.

7.4 BaseModel — Defining Data Schemas

`BaseModel` is provided by `Pydantic` and serves as the foundation for defining the structure of data that your API accepts or returns. By inheriting from `BaseModel`, you define a class with typed fields, and `Pydantic` will automatically validate incoming data against this schema.

```
from fastapi import FastAPI
from pydantic import BaseModel

class Student(BaseModel):
    id: int = None      # Optional field (None by default)
    name: str           # Required string field
    age: int            # Required integer field
    course: str         # Required string field

app = FastAPI()
students = []          # In-memory storage (list of dicts)
```

Important Note

In production applications, the in-memory list would be replaced by a database such as MongoDB, PostgreSQL, or MySQL. The in-memory approach is used here for simplicity and clarity during learning.

Chapter 8: CRUD Operations in FastAPI

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations in any data management system. In REST APIs, these operations correspond directly to HTTP methods:

CRUD Operation	HTTP Method	Description
Create	POST	Send data to the server to create a new resource
Read	GET	Retrieve data from the server
Update	PUT / PATCH	Modify an existing resource on the server
Delete	DELETE	Remove a resource from the server

8.1 GET — Root and Get All

GET endpoints retrieve data from the server. They should not have side effects — meaning calling a GET endpoint multiple times should return the same result without changing server state.

```
# Root endpoint - health check / welcome message
@app.get("/")
def home():
    return {"msg": "FastAPI calling"}

# Get all students
@app.get("/getstudents")
def get_students():
    return students

# Postman: GET http://127.0.0.1:8000/getstudents
```

8.2 POST — Create a New Record

POST endpoints receive data in the request body (typically JSON) and use it to create a new resource. FastAPI automatically parses and validates the JSON body against the Pydantic model.

```
@app.post("/studentpost")
def studentpost(student: Student):
    students.append(student.dict()) # convert model to dict
    return {
        "msg": "Student added successfully",
        "data": student
    }

# Postman: POST http://127.0.0.1:8000/studentpost
# Body (raw JSON):
# { "id": 1, "name": "Kenvi", "age": 23, "course": "MERN Stack" }
```

8.3 GET by ID — Path Parameters

Path parameters are variable parts of the URL path. They allow the client to specify which specific resource to retrieve. Path parameters are defined using curly braces in the route decorator.

```
@app.get("/studentgetbyid/{id}")
def get_id(id: int):
    for student in students:
        if student["id"] == id:
            return student
    return {"msg": "No student found"}
# Postman: GET http://127.0.0.1:8000/studentgetbyid/1
```

8.4 PUT — Update an Existing Record

PUT endpoints update an entire existing resource. The client sends new data for all fields, which replaces the existing record. FastAPI receives both the path parameter (identifying the resource) and the request body (the new data).

```
@app.put("/Putstudent/{id}")
def update(id: int, up_stu: Student):
    for student in students:
        if student["id"] == id:
            student["id"] = up_stu.id
            student["name"] = up_stu.name
            student["age"] = up_stu.age
            student["course"] = up_stu.course
            return {"msg": "Student updated", "data": student}
    return {"msg": "Student not found"}
```

8.5 DELETE — Hard and Soft Delete

DELETE endpoints remove resources. In practice, there are two types of deletion: hard delete (permanently removes the record) and soft delete (marks the record as deleted but keeps it in the database for audit purposes).

```
# Hard delete by ID
@app.delete("/deletestudent/{id}")
def delete_student(id: int):
    for student in students:
        if student["id"] == id:
            students.remove(student)
            return {"msg": "Deleted", "data": student}
    return {"msg": "Not found"}
```

```
# Soft delete - flag as deleted, preserve data
@app.delete("/softdelete/{id}")
def softdelete(id: int):
    for student in students:
        if student["id"] == id:
            student["is_deleted"] = True
            return {"msg": "Soft deleted", "data": student}
    return {"msg": "Not found"}
```

Hard vs Soft Delete

Soft delete is strongly preferred in production systems. It preserves historical data for auditing, allows recovery from accidental deletions, and maintains referential integrity in relational databases. Hard delete is used only when data must be permanently removed (e.g., GDPR compliance requests).

Chapter 9: Status Codes, Query Parameters & API Documentation

9.1 HTTP Status Codes

HTTP status codes are three-digit numbers returned by the server with every response. They tell the client whether the request was successful, caused an error, or requires further action. FastAPI uses status codes from the Python http module and allows setting custom codes per endpoint.

Status Code	Meaning	When to Use
200 OK	Request succeeded	Successful GET, PUT, DELETE
201 Created	Resource created successfully	Successful POST
400 Bad Request	Client sent invalid data	Validation failure, missing fields
401 Unauthorized	Authentication required	Missing or invalid token
403 Forbidden	Access denied	Valid auth but insufficient permissions
404 Not Found	Resource does not exist	ID not found in database
422 Unprocessable	Validation error	FastAPI auto-returns on Pydantic failure
500 Internal Server Error	Unexpected server error	Unhandled exception in server code

```
from fastapi import FastAPI, Response

@app.post("/student")
async def add_student(response: Response):
    response.status_code = 201
    return {"message": "Student Added"}
```

9.2 Query Parameters, Path Parameters, and Request Body

FastAPI supports three primary ways to receive data from a client, each suited to different scenarios:

Method	Location in Request	HTTP Methods	Example	Best Used For
--------	---------------------	--------------	---------	---------------

Method	Location in Request	HTTP Methods	Example	Best Used For
Query Parameters	URL after ?	GET	?name=meet&age=22	Filtering, searching, pagination
Path Parameters	Inside URL path	GET, DELETE	/users/10	Identifying a specific resource
Request Body	HTTP body (JSON)	POST, PUT	JSON payload	Sending complete data objects

```

from fastapi import FastAPI
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

# Combines all three in one endpoint
@app.post("/user/{id}")
def get_user(id: int, city: str, user: User):
    return {"id": id, "city": city, "user": user}
# id = path parameter (from URL)
# city = query parameter (from ?city=Surat)
# user = request body (from JSON)

```

9.3 Swagger UI and Postman

FastAPI automatically generates interactive API documentation without any additional code:

- Swagger UI: Available at <http://127.0.0.1:8000/docs> — allows testing all endpoints directly from the browser with a visual interface.
- ReDoc: Available at <http://127.0.0.1:8000/redoc> — provides a cleaner, read-focused alternative documentation view.
- Postman: A popular desktop API testing tool that provides more flexibility for complex testing scenarios, automated test suites, and team collaboration.

9.3.1 Using Postman for API Testing

40. Open Postman and create a new request.
41. Select the HTTP method (GET, POST, PUT, DELETE).
42. Enter the request URL (e.g., <http://127.0.0.1:8000/studentpost>).
43. For POST/PUT: Click Body > raw > JSON and enter the JSON payload.
44. Click Send and review the response body and status code.

Chapter 10: Practice Questions and Exercises — Day 2

10.1 Short Answer Questions

45. What does REST stand for? What are the key principles of REST architecture?
46. What is the difference between FastAPI, Flask, and Django? When would you choose FastAPI?
47. What is Pydantic's BaseModel? Why is it important in FastAPI?
48. What is the difference between a path parameter and a query parameter?
49. What does the `--reload` flag do when running uvicorn?
50. What is the HTTP status code for 'Resource Created Successfully'?
51. What is the difference between hard delete and soft delete? Give a use case for each.
52. What is Swagger UI? How do you access it in a FastAPI application?
53. What does the `.dict()` method do on a Pydantic model?
54. What is an ASGI server? Why is uvicorn used with FastAPI?

10.2 Long Answer Questions

55. Explain the complete anatomy of a FastAPI application from installation to a running CRUD API. Include all required imports, model definition, route decorators, and testing steps.
56. Compare GET, POST, PUT, and DELETE HTTP methods in detail. For each method, explain: its purpose, whether it sends a body, idempotency, and a practical example in FastAPI.
57. Explain how Pydantic performs data validation in FastAPI. What happens when a client sends invalid data? What is a 422 error?
58. Describe the concept of RESTful API design principles (Statelessness, Uniform Interface, Client-Server, etc.) and explain how FastAPI adheres to or supports each principle.

10.3 Practical / Coding Questions

59. Build a complete FastAPI application for a Book Library with the following endpoints: POST `/books` (add a book), GET `/books` (get all books), GET `/books/{id}` (get one book), PUT `/books/{id}` (update book), DELETE `/books/{id}` (hard delete).
60. Extend the student API to include a GET endpoint with a query parameter that filters students by course name: GET `/students?course=MERN`
61. Add a soft delete feature to the student API. Add an `is_active` field to the Student model (default True). The soft delete endpoint should set `is_active` to False. The get all students endpoint should only return active students.

62. Add proper HTTP status codes to each endpoint of the student API using FastAPI's Response object or status_code parameter on the decorator.

10.4 Multiple Choice Questions

63. Which HTTP method is used to create a new resource? a) GET b) PUT c) POST d) DELETE Answer: c) POST
64. Where are query parameters located in an HTTP request? a) In the request body b) In the URL after ? c) In headers d) In path segments Answer: b) In the URL after ?
65. What is the default port for a FastAPI application running on uvicorn? a) 3000 b) 5000 c) 8000 d) 8080 Answer: c) 8000
66. Which library provides data validation and type enforcement in FastAPI? a) SQLAlchemy b) Marshmallow c) Pydantic d) Cerberus Answer: c) Pydantic
67. At which URL is the auto-generated Swagger documentation available in FastAPI? a) /swagger b) /api-docs c) /docs d) /help Answer: c) /docs

10.5 Interview Questions

68. What is the difference between synchronous and asynchronous endpoints in FastAPI? When should you use async def?
69. How does FastAPI handle request validation errors? What does the 422 response look like?
70. What is CORS? How do you enable it in FastAPI?
71. How would you add JWT authentication to a FastAPI application?
72. What is dependency injection in FastAPI? Give a practical example.

MODULE 3

Day 3: Generative AI, Prompt Engineering & LangChain

Concepts, LLM Calling, Prompt Techniques & Application Development

Chapter 11: Introduction to Generative AI

11.1 What is Generative AI?

Generative AI is a branch of Artificial Intelligence that focuses on creating systems capable of generating entirely new content — text, images, audio, video, code, and more — based on patterns learned from large datasets during training.

Unlike traditional AI systems that are designed to classify, predict, or make decisions based on existing data, Generative AI models produce original output that did not previously exist. This is accomplished through deep learning architectures — particularly Transformer models — that learn the statistical distributions and relationships within massive training datasets.

Key Insight

Generative AI answers the question: 'What should come next?' rather than 'What category does this belong to?' This fundamental shift in objective is what enables the creation of novel content.

11.2 Types of Content Generated by Generative AI

Content Type	Description	Leading Models
Text	Essays, summaries, code, answers, emails, stories	GPT-4, Claude, Gemini
Images	Artwork, illustrations, logos, photorealistic images	DALL-E, Midjourney, Stable Diffusion
Video	Short clips, animations, deepfakes, avatars	Sora, Runway, Pika
Code	Functions, programs, bug fixes, test cases	GitHub Copilot, CodeWhisperer
Audio	Music generation, voice synthesis, sound effects	MusicGen, Eleven Labs, Suno

11.3 Traditional AI vs. Generative AI

Dimension	Traditional AI	Generative AI
Primary Goal	Classify, predict, or decide	Create new, original content
Output Type	Label, score, category, or decision	Text, image, video, or code

Dimension	Traditional AI	Generative AI
Example Systems	Spam filter, face recognition, fraud detection	ChatGPT, DALL-E, GitHub Copilot
Learning Approach	Learn decision boundaries from labeled data	Learn probability distributions from data
Key Question	'What category is this?'	'What should come next?'

11.4 How Generative AI Works — The Complete Workflow

Understanding how Generative AI processes a request from start to finish helps developers build better prompts and applications.

- 73. **Input Prompt:** The user types a question, instruction, or description. This is the starting point for the entire generation process.
- 74. **Tokenization:** The text is broken down into tokens — small chunks of text (words, sub-words, or characters). The model processes tokens rather than raw characters. Example: 'Hello world' becomes [15339, 1917] in GPT-3.5's token vocabulary.
- 75. **Neural Network Processing:** The tokenized input is processed through dozens (or hundreds) of Transformer layers. Each layer refines the model's understanding of context and meaning using the Attention mechanism.
- 76. **Generation:** The model predicts the next most likely token, one at a time, based on all previous context. This is repeated until the response is complete or a stop condition is reached.
- 77. **Output:** The generated tokens are decoded back into human-readable text and returned to the user.

11.4.1 Key Technical Concepts

Concept	Description
Transformer	Neural network architecture that processes all tokens simultaneously using self-attention, enabling understanding of long-range relationships in text.
Attention Mechanism	Allows the model to dynamically focus on the most relevant parts of the input when generating each output token.
Temperature	Controls randomness in generation. Low (0.1): focused, deterministic answers. High (1.0+): creative, varied, unpredictable responses.
Context Window	The maximum amount of text the model can 'see' at once — its working memory. Ranges from 4K tokens (older models) to 200K+ tokens (modern models like Claude).
Token	The basic unit of text the model processes. Roughly 1 token = 0.75 English words. Pricing for API usage is typically based on tokens.

11.5 The AI Hierarchy — AI, ML, DL, and Generative AI

Level	Full Name	Definition	Key Technique	Example
AI	Artificial Intelligence	Machines that simulate human intelligence	Rules, logic, statistics	Chatbots, search engines
ML	Machine Learning	Systems that learn from data automatically	Regression, decision trees	Spam filter, Netflix recommendations
DL	Deep Learning	ML using multi-layer neural networks	CNNs, RNNs, LSTMs	Face recognition, voice assistants
Gen AI	Generative AI	DL that creates new content	Transformers, Diffusion	ChatGPT, DALL-E, Copilot

Each level is a subset of the one above it. Generative AI represents the cutting edge of AI development, built on the full power of deep neural networks.

Chapter 12: Prompt Engineering

12.1 What is Prompt Engineering?

Prompt Engineering is the discipline of designing, crafting, and refining instructions (prompts) given to an AI model to obtain the most accurate, relevant, and useful responses. As AI models become more capable, the ability to communicate effectively with them has become a valuable professional skill.

The quality of output from a Generative AI model is directly proportional to the quality of the input prompt. A well-engineered prompt can transform a vague, unhelpful AI response into a precise, actionable, and professional output.

Simple Analogy

Prompt engineering is like giving clear instructions to a highly capable employee. Vague: 'Do the report.' Better: 'Write a 2-page sales performance report for Q3, comparing regions, identifying top performers, and suggesting 3 improvement strategies. Use a professional tone for a board audience.'

12.2 Types of Prompts

Type	Description	Example
Good Prompt	Clear, specific, provides sufficient context	"Explain Python loops to a beginner with a simple example."
Bad Prompt	Vague, too short, missing context	"Tell me about Python." (too broad)
Clear Prompt	Simple, direct language with no ambiguity	"What is a Python dictionary? Give a one-line definition."
Detailed Prompt	Includes role, context, format, and constraints	"Act as a Python tutor. Explain list comprehensions to a 10th-grade student. Use 2 real-world examples, under 100 words."

12.3 The Anatomy of a Great Prompt

A high-quality prompt typically includes five key components:

Component	Purpose	Example
Role	Defines the persona the AI	"Act as a senior Python developer"

Component	Purpose	Example
	should adopt	
Context	Provides background information or constraints	"I know JavaScript and want to learn Python"
Task	Specifies exactly what the AI should do	"Explain Python list comprehensions"
Constraints	Sets limitations or rules for the response	"Use simple language with examples, max 200 words"
Output Format	Defines the format of the desired response	"Provide the answer in a numbered list with code samples"

12.3.1 Low Quality vs High Quality Prompt Example

Quality	Prompt
Low Quality	"Write code."
High Quality	"Act as a Python developer. Write a function that accepts a list of student dictionaries with 'name' and 'score' fields, sorts them by score in descending order, and returns the top 3. Add inline comments explaining each step. Use Python 3.10+ syntax."

12.4 Prompting Techniques

12.4.1 Zero-Shot Prompting

Zero-shot prompting asks the AI to perform a task without providing any examples. The model relies entirely on its pre-trained knowledge. This is the most common prompting approach and works well for general, well-defined tasks.

```
Prompt: "Translate 'I love programming' to French."
Output: "J'aime la programmation."
```

12.4.2 One-Shot Prompting

One-shot prompting provides exactly one example before the actual task. This helps the model understand the expected pattern, format, or style of the response.

```
Prompt:
English: Hello → French: Bonjour
English: Thank you → French: ?

Output: Merci
```

12.4.3 Few-Shot Prompting

Few-shot prompting provides multiple examples (typically 3–5) before the task. More examples lead to more consistent, accurate, and appropriately formatted outputs. This technique is especially useful for custom output formats and pattern matching.

```
Prompt:
Sentence: The movie was fantastic! → Sentiment: Positive
Sentence: I hated the food.          → Sentiment: Negative
Sentence: The service was okay.      → Sentiment: Neutral
Sentence: This is the best day ever! → Sentiment: ?

Output: Positive
```

12.4.4 Chain of Thought (CoT) Prompting

Chain of Thought prompting encourages the AI to reason through problems step by step rather than jumping directly to an answer. It significantly improves accuracy on mathematical, logical, and multi-step problems. The trigger phrase 'Let's think step by step' is commonly used.

```
# Without CoT
Prompt: "A shop has 12 apples. Sells 5, buys 8. Total?"
Output: 15 (may guess incorrectly)

# With Chain of Thought
Prompt: "A shop has 12 apples. Sells 5, buys 8.
        Let's think step by step."
Output:
Step 1: Initial apples = 12
Step 2: Sold = 5, Remaining = 12 - 5 = 7
Step 3: Bought = 8, Total = 7 + 8 = 15
Final Answer: 15
```

12.4.5 Prompting Techniques Comparison

Technique	Examples Provided	Best Used For	Memory Aid
Zero-Shot	None	General tasks, translation, Q&A	Do directly
One-Shot	1 example	Format learning, pattern matching	Learn from ONE
Few-Shot	2–5 examples	Complex patterns, structured output	Learn from MANY
Chain of Thought	Step-by-step reasoning	Math, logic, multi-step problems	Think step by step

Chapter 13: AI Terminology, Embeddings, and Tokenization

13.1 Essential AI Terminology

Term	Definition
Artificial Intelligence (AI)	Technology enabling machines to simulate human intelligence, reasoning, and decision-making.
Machine Learning (ML)	Branch of AI where systems learn patterns from data without being explicitly programmed for each task.
Neural Network	Computing model loosely inspired by the brain, consisting of interconnected nodes (neurons) organized in layers.
Generative AI (Gen AI)	A class of AI that can generate new content — text, images, audio, or code — based on patterns in training data.
Large Language Models (LLMs)	AI models trained on vast amounts of text data to understand and generate human-like language (e.g., GPT-4, Claude, Gemini).
Transformer	The neural network architecture that powers modern LLMs; uses attention mechanisms to process all tokens simultaneously.
Fine-tuning	Process of further training a pre-trained model on a smaller, domain-specific dataset to specialize its behavior.
Hallucination	When an AI model generates information that is factually incorrect or fabricated, presented with apparent confidence.
Inference	The process of using a trained model to generate responses (as opposed to training the model).

13.2 Vector Embeddings

Vector embeddings are numerical representations of text (or other data) as arrays of floating-point numbers. They are the fundamental mechanism through which AI models understand meaning and semantic relationships between words, sentences, or documents.

The key insight is that words or concepts with similar meanings have embeddings that are mathematically close to each other in vector space. This allows AI models to compare, search, cluster, and reason about text based on semantic similarity rather than exact character matching.

```
# Conceptual example of word embeddings
"King"  → [0.21, 0.98, 0.45, -0.12, 0.67, ...]
"Queen" → [0.23, 0.95, 0.47, -0.10, 0.65, ...]
"Dog"   → [0.91, 0.12, -0.34, 0.78, -0.23, ...]
```

```
# King and Queen are mathematically close (similar meaning)
# Dog is far from both (different semantic space)
```

Embeddings power applications such as semantic search (find documents by meaning, not keywords), recommendation systems, clustering similar documents, and Retrieval-Augmented Generation (RAG).

13.3 Tokens and Tokenization

A token is the basic unit of text that language models process. Tokens can be whole words, parts of words, or punctuation marks. Models do not think in characters or complete words — they think in tokens.

13.3.1 Using tiktoken (OpenAI's Tokenizer)

```
# pip install tiktoken
import tiktoken

enc = tiktoken.encoding_for_model("gpt-3.5-turbo")

# Encode: text -> token IDs
tokens = enc.encode("Hello world")
print(tokens)          # [15339, 1917]

# Decode: token IDs -> text
text = enc.decode(tokens)
print(text)            # 'Hello world'

# Count tokens in a string
text = "This is a sample sentence for token counting."
print(len(enc.encode(text))) # 9 tokens
```

Practical Note on Token Pricing

API calls to LLMs are priced per token (both input and output tokens). Understanding tokenization helps estimate costs, optimize prompt length, and stay within context window limits. As a rough guide: 1 token $\approx$ 0.75 English words, or 1,000 tokens $\approx$ 750 words.

Chapter 14: Calling Large Language Models from Python

14.1 Setup and Authentication

To call an LLM (such as GPT-3.5 or GPT-4) programmatically, you need an API key from the provider. Best practice is to store API keys in a `.env` file and load them using the `python-dotenv` library, rather than hardcoding them in source code.

```
# Install required packages
# pip install openai python-dotenv

# .env file (never commit to version control)
# OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

14.2 Basic LLM Call — Single Turn

A basic API call to an LLM sends a list of messages and receives a completion response. The messages array supports three roles: system (highest-priority instructions), user (user input), and assistant (previous AI responses for context).

```
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()

client = OpenAI()
user_input = input("User: ")

response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a world travel guide."},
        {"role": "user", "content": user_input}
    ]
)

print(response.choices[0].message.content)
```

14.3 Multi-Turn Conversation with Memory

Maintaining conversational context requires sending the full message history with each API call. The model has no built-in memory between calls — context must be managed explicitly by the developer.

```
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()

client = OpenAI()
messages = [
    {"role": "system", "content": "You are a world travel guide."},
]

for i in range(3):
    user_input = input("User: ")
    messages.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
        temperature=0.9,
    )
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    print(reply)
```

14.4 LLM Parameters Explained

Parameter	What It Controls	Recommended Range
temperature	Controls randomness/creativity. Low = focused/deterministic. High = creative/varied.	0.0–0.3 for factual; 0.7–1.0 for creative
top_p	Nucleus sampling — restricts model to top-p% probability mass tokens.	0.9–1.0 for most tasks
top_k	Restricts model to choosing from only the top K probable next tokens.	40–100 for balanced output
max_tokens	Maximum number of tokens in the generated response.	Depends on expected output length

Chapter 15: Building AI Applications with LangChain

15.1 What is LangChain?

LangChain is an open-source framework designed to simplify the development of applications powered by Large Language Models. Rather than writing raw API calls and managing complex orchestration logic manually, LangChain provides high-level abstractions called chains, which allow developers to compose multiple AI-powered components together using a clean, expressive syntax.

15.1.1 Core Concepts

- LCEL (LangChain Expression Language): Uses the pipe operator (`|`) to chain components, similar to Unix pipes.
- PromptTemplates: Reusable prompt structures with variable placeholders.
- LLM Wrappers: Unified interface to multiple LLM providers (OpenAI, Anthropic, Google, etc.).
- Output Parsers: Convert raw LLM text responses into structured Python objects.
- Memory: Mechanisms for maintaining conversation history across multiple turns.
- Agents: Autonomous AI systems that can use tools and make decisions.

```
# Installation
pip install langchain langchain-openai python-dotenv
```

15.2 Basic LangChain Chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

# 1. Initialize the LLM
llm = ChatOpenAI(model="gpt-3.5-turbo")

# 2. Create a reusable Prompt Template
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert educator. Add at least 5 questions."),
    ("user", "Create a question paper on {topic}, {marks} marks, {difficulty} difficulty."),
])

# 3. Build chain using LCEL pipe syntax
chain = prompt | llm
```

```
# 4. Invoke with variables
response = chain.invoke({
    "topic":      "Python programming",
    "marks":      100,
    "difficulty": "medium"
})
print(response.content)
```

15.3 LLM Streaming

Streaming allows displaying LLM responses token-by-token as they are generated, rather than waiting for the complete response. This dramatically improves perceived performance and is the standard approach in production chatbot applications.

```
response = chain.stream({
    "topic":      "Python programming",
    "marks":      100,
    "difficulty": "medium"
})

for chunk in response:
    print(chunk.content, end="", flush=True)
```

15.4 Output Parsers

Output parsers transform the raw text output from an LLM into structured Python data types. LangChain provides three main built-in parsers:

15.4.1 StrOutputParser — Plain Text

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()
chain = prompt | llm | parser
result = chain.invoke({...})
print(result)    # Plain text string (not AIMessage object)
```

15.4.2 JsonOutputParser — Python Dictionary

```
from langchain_core.output_parsers import JsonOutputParser

parser = JsonOutputParser()
```

```
# Important: prompt must instruct the model to return JSON
chain = prompt | llm | parser
result = chain.invoke({...})
print(result)    # Python dict, fully parsed
print(result["questions"])    # Access like normal dict
```

15.4.3 PydanticOutputParser — Strongly Typed Structured Output

The most powerful parser — validates the output against a Pydantic schema, providing type safety and structured data access.

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Question(BaseModel):
    question: str = Field(description="Question text")
    options: list[str] = Field(description="4 answer options")
    correct_answer: str = Field(description="The correct option")

class QuestionPaper(BaseModel):
    topic: str
    marks: int
    difficulty: str
    questions: list[Question]
    passing_marks: int

parser = PydanticOutputParser(pydantic_object=QuestionPaper)

# Include format_instructions in system prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert educator.\n{format_instructions}"),
    ("user", "Create a question paper on {topic}, {marks} marks, {difficulty}."),
])

chain = prompt | llm | parser
response = chain.invoke({
    "topic": "Python",
    "marks": 100,
    "difficulty": "medium",
    "format_instructions": parser.get_format_instructions()
})
print(response.model_dump_json(indent=2))
```

15.5 Memory in LangChain

By default, LLMs are stateless — they have no memory of previous conversations. LangChain provides several memory modules to manage conversational context:

Memory Type	What It Stores	Ideal Use Case
ConversationBufferMemory	All messages, in full	Short conversations where full context matters
ConversationBufferWindowMemory (k)	Only the last k exchanges	Long conversations; saves tokens, minor context loss
ConversationSummaryMemory	AI-generated running summary	Very long conversations; minimal token usage
ConversationSummaryBufferMemory	Recent messages + summary of older	Balance between accuracy and token efficiency
Database Memory	Persisted in external DB (Redis, MongoDB)	Multi-session apps, production chatbots

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
memory.save_context({"input": "Hi"}, {"output": "Hello!"})
memory.save_context({"input": "My name is Alice"}, {"output": "Nice to meet you, Alice!"})

print(memory.load_memory_variables({}))
# {'history': 'Human: Hi\nAI: Hello!\nHuman: My name is Alice\nAI: Nice to meet you, Alice!'}
```


Chapter 16: Practice Questions and Exercises — Day 3

16.1 Short Answer Questions

78. What is Generative AI? How is it different from traditional AI?
79. What is a token in the context of language models?
80. Explain the difference between zero-shot and few-shot prompting.
81. What is Chain of Thought prompting? When is it most useful?
82. What are vector embeddings? Why are they important in AI?
83. What is the purpose of the 'system' role in an LLM API message?
84. What is the temperature parameter? What happens at temperature=0?
85. What is LangChain? What problem does it solve?
86. What is LCEL? How is the pipe operator used in LangChain?
87. Describe the five types of memory available in LangChain.

16.2 Long Answer Questions

88. Explain the complete workflow of a Generative AI model — from user input to final output. Include: tokenization, Transformer architecture, attention mechanism, generation process, and decoding. Provide an illustrative diagram description.
89. Describe the AI hierarchy (AI > ML > DL > Gen AI > LLMs). For each level, explain the definition, key techniques, and provide two real-world examples.
90. Explain prompt engineering in detail. Cover: definition, importance, the five components of a great prompt, and all four prompting techniques (Zero-Shot, One-Shot, Few-Shot, Chain of Thought). Include an example for each technique.
91. Explain LangChain's output parsers in detail. Compare StrOutputParser, JsonOutputParser, and PydanticOutputParser with code examples. When would you use each?
92. Describe all five memory types in LangChain. Explain what each stores, its advantages and limitations, and when you would choose each type in a real application.

16.3 Practical / Coding Questions

93. Write a Python script that calls the OpenAI API to build a multi-turn chatbot. The bot should maintain the full conversation history and support at least 5 turns of conversation.
94. Write a LangChain chain that accepts a topic, difficulty, and number of questions, then generates a structured quiz using PydanticOutputParser. The output should be a Python object with a list of Question objects, each having a question, four options, and a correct answer.

95. Build a LangChain application that uses ConversationBufferWindowMemory (k=3) to maintain only the last 3 exchanges. Demonstrate how old messages are dropped from context.
96. Write a Python program using tiktoken to count the tokens in different sentences and identify which sentences would exceed a 50-token limit.
97. Create a LangChain chain that takes a Python code snippet and returns a JSON object with: a description of what the code does, potential bugs, and suggestions for improvement.

16.4 Multiple Choice Questions

98. Which component of an LLM decides how much to 'focus' on different parts of the input? a) Embedding layer b) Attention mechanism c) Tokenizer d) Softmax layer Answer: b) Attention mechanism
99. What does a high temperature value (e.g., 1.2) produce? a) More focused and deterministic output b) Shorter responses c) More creative and varied output d) Faster response times Answer: c) More creative and varied output
100. Which prompting technique involves showing the model step-by-step reasoning? a) Zero-Shot b) One-Shot c) Few-Shot d) Chain of Thought Answer: d) Chain of Thought
101. In LangChain's LCEL syntax, what operator is used to chain components? a) >> b) + c) | d) -> Answer: c) |
102. Which memory type in LangChain keeps only the most recent K messages? a) ConversationBufferMemory b) ConversationSummaryMemory c) ConversationBufferWindowMemory d) Database Memory Answer: c) ConversationBufferWindowMemory
103. What does tokenization do in a language model? a) Converts text to images b) Breaks text into small units for processing c) Measures semantic similarity d) Encrypts API keys Answer: b) Breaks text into small units for processing

16.5 Scenario-Based Questions

104. Scenario: You are building a customer support chatbot for a bank. The chatbot needs to answer account-related questions accurately and consistently. Which temperature setting would you use and why? Which memory type would you choose for a session lasting 50+ messages? Design the system prompt for this chatbot.
105. Scenario: Your team needs to build an AI tool that automatically generates structured employee review reports from raw performance notes. The output must be a JSON object with specific fields (strengths, areas_for_improvement, rating, recommended_actions). Design the LangChain chain for this, including the Pydantic model and output parser.
106. Scenario: A user asks the AI: 'What is 15% of 847, and is it odd or even?' Without Chain of Thought prompting, the AI gives a wrong answer. Rewrite the prompt to include CoT reasoning and predict the correct step-by-step output.

16.6 Interview Questions

107. What is Retrieval-Augmented Generation (RAG)? How does it work and what problem does it solve?
108. Explain the difference between fine-tuning an LLM and using prompt engineering. When would you use each approach?
109. What is a hallucination in AI? How can prompt engineering reduce hallucinations?
110. What is the context window of a language model? Why is it important for application design?
111. What are AI Agents in LangChain? How do they differ from simple chains?
112. Explain the concept of embeddings and similarity search. How would you build a semantic search engine using Python?

Appendix A: Quick Reference — Python Cheat Sheet

A.1 Data Structure Methods Quick Reference

Structure	Key Methods
List	append, insert, remove, pop, sort, reverse, index, count, copy, clear
Tuple	count, index
Set	add, update, remove, discard, union, intersection, difference, symmetric_difference
Dictionary	keys, values, items, get, update, pop, clear, copy
String	upper, lower, strip, replace, split, join, find, startswith, endswith, count

A.2 FastAPI HTTP Methods Reference

Method	Decorator	Purpose
GET	@app.get('/path')	Retrieve data
POST	@app.post('/path')	Create new data
PUT	@app.put('/path/{id}')	Update existing data
DELETE	@app.delete('/path/{id}')	Delete data

A.3 LangChain Output Parsers Quick Reference

Parser	Output Type	Use When
StrOutputParser	Plain string	Simple text responses, no structure needed
JsonOutputParser	Python dict	Flexible JSON needed, schema may vary
PydanticOutputParser	Pydantic model instance	Strict schema required, type safety needed

A.4 Prompting Quick Reference Card

Technique	Use For	Trigger Phrase
Zero-Shot	General tasks	(none needed)
One-Shot	Format matching	"Example: ... → Output: ..."

Technique	Use For	Trigger Phrase
Few-Shot	Consistency	"Example 1:... Example 2:..."
Chain of Thought	Math & logic	"Let's think step by step."

Appendix B: Glossary of Terms

Term	Definition
API	Application Programming Interface — a set of rules enabling software components to communicate.
ASGI	Asynchronous Server Gateway Interface — the web server interface used by FastAPI (uvicorn).
Attention	Mechanism in Transformers allowing the model to focus on relevant parts of the input.
BaseModel	Pydantic class used to define data schemas for validation in FastAPI.
Chain	A sequence of LangChain components connected via the pipe () operator.
Context Window	The maximum number of tokens an LLM can process in a single call.
CRUD	Create, Read, Update, Delete — the four fundamental data operations.
DL (Deep Learning)	ML using multi-layer neural networks that can learn complex feature representations.
Embedding	Numerical vector representation of text that captures semantic meaning.
F-String	Python string formatting syntax: f"Hello, {name}" — available from Python 3.6+.
Gen AI	Generative AI — AI systems capable of creating new content from patterns in training data.
Hallucination	When an AI generates confidently stated but factually incorrect information.
HTTP	HyperText Transfer Protocol — the communication protocol underlying REST APIs.
Immutable	An object whose value cannot be changed after creation (e.g., strings, tuples).
JSON	JavaScript Object Notation — lightweight text format for data exchange between systems.
Lambda	Anonymous Python function defined with the lambda keyword for short inline operations.
LCEL	LangChain Expression Language — uses the pipe operator to chain AI components.
LLM	Large Language Model — AI trained on vast text data to generate human-like language.
ML (Machine Learning)	AI systems that learn patterns from data without explicit rule programming.
Mutable	An object whose value can be changed after creation (e.g., lists, sets, dicts).
Pydantic	Python library for data validation using type hints; core component of FastAPI.

Term	Definition
REST	Representational State Transfer — architectural style for designing networked APIs.
Soft Delete	Marking a record as deleted (with a flag) without removing it from the database.
Temperature	LLM parameter controlling output randomness (0=deterministic, 1+=creative).
Token	The basic unit of text processed by language models; approximately 0.75 English words.
Tokenization	The process of converting text into tokens before processing by a language model.
Transformer	Neural network architecture powering modern LLMs; uses self-attention mechanisms.
Type Hint	Python syntax to declare expected data types: name: str, age: int.
Uvicorn	High-performance ASGI web server used to run FastAPI applications.
Vector	Mathematical array of numbers used to represent embeddings in semantic space.

End of Training Manual

Python & Artificial Intelligence — 3-Day Intensive Course

This manual covers: Core Python Fundamentals | FastAPI REST APIs | Generative AI |
LangChain